compact1, compact2, compact3

java.util.concurrent.locks

Class ReentrantReadWriteLock

java.lang.Object
  java.util.concurrent.locks.ReentrantReadWriteLock

All Implemented Interfaces:

Serializable, ReadWriteLock

```
public class ReentrantReadWriteLock
extends Object
implements ReadWriteLock, Serializable
```

An implementation of ReadWriteLock supporting similar semantics to ReentrantLock.

This class has the following properties:

- **Acquisition order**

  This class does not impose a reader or writer preference ordering for lock access. However, it does support an optional *fairness* policy.

  Non-fair mode (default)

  When constructed as non-fair (the default), the order of entry to the read and write lock is unspecified, subject to reentrancy constraints. A nonfair lock that is continuously contended may indefinitely postpone one or more reader or writer threads, but will normally have higher throughput than a fair lock.

  Fair mode

  When constructed as fair, threads contend for entry using an approximately arrival-order policy. When the currently held lock is released, either the longest-waiting single writer thread will be assigned the write lock, or if there is a group of reader

threads waiting longer than all waiting writer threads, that group will be assigned the read lock.

A thread that tries to acquire a fair read lock (non-reentrantly) will block if either the write lock is held, or there is a waiting writer thread. The thread will not acquire the read lock until after the oldest currently waiting writer thread has acquired and released the write lock. Of course, if a waiting writer abandons its wait, leaving one or more reader threads as the longest waiters in the queue with the write lock free, then those readers will be assigned the read lock.

A thread that tries to acquire a fair write lock (non-reentrantly) will block unless both the read lock and write lock are free (which implies there are no waiting threads). (Note that the non-blocking `ReentrantReadWriteLock.ReadLock.tryLock()` and `ReentrantReadWriteLock.WriteLock.tryLock()` methods do not honor this fair setting and will immediately acquire the lock if it is possible, regardless of waiting threads.)

- **Reentrancy**

  This lock allows both readers and writers to reacquire read or write locks in the style of a `ReentrantLock`. Non-reentrant readers are not allowed until all write locks held by the writing thread have been released.

  Additionally, a writer can acquire the read lock, but not vice-versa. Among other applications, reentrancy can be useful when write locks are held during calls or callbacks to methods that perform reads under read locks. If a reader tries to acquire the write lock it will never succeed.

- **Lock downgrading**

  Reentrancy also allows downgrading from the write lock to a read lock, by acquiring the write lock, then the read lock and then releasing the write lock. However, upgrading from a read lock to the write lock is **not** possible.

- **Interruption of lock acquisition**

  The read lock and write lock both support interruption during lock acquisition.

- **`Condition` support**

  The write lock provides a `Condition` implementation that behaves in the same way, with respect to the write lock, as the `Condition` implementation provided by `ReentrantLock.newCondition()` does for `ReentrantLock`. This `Condition` can, of course, only be used with the write lock.

  The read lock does not support a `Condition` and `readLock().newCondition()` throws `UnsupportedOperationException`.

- **Instrumentation**

> This class supports methods to determine whether locks are held or contended. These methods are designed for monitoring system state, not for synchronization control.

Serialization of this class behaves in the same way as built-in locks: a deserialized lock is in the unlocked state, regardless of its state when serialized.

Sample usages. Here is a code sketch showing how to perform lock downgrading after updating a cache (exception handling is particularly tricky when handling multiple locks in a non-nested fashion):

```
class CachedData {
  Object data;
  volatile boolean cacheValid;
  final ReentrantReadWriteLock rwl = new ReentrantReadWriteLock();

  void processCachedData() {
    rwl.readLock().lock();
    if (!cacheValid) {
      // Must release read lock before acquiring write lock
      rwl.readLock().unlock();
      rwl.writeLock().lock();
      try {
        // Recheck state because another thread might have
        // acquired write lock and changed state before we did.
        if (!cacheValid) {
          data = ...
          cacheValid = true;
        }
        // Downgrade by acquiring read lock before releasing write lock
        rwl.readLock().lock();
      } finally {
        rwl.writeLock().unlock(); // Unlock write, still hold read
      }
    }

    try {
      use(data);
    } finally {
      rwl.readLock().unlock();
    }
  }
}
```

ReentrantReadWriteLocks can be used to improve concurrency in some uses of some kinds of Collections. This is typically worthwhile only when the collections are expected to be large, accessed by more reader threads than writer threads, and entail operations with overhead that outweighs synchronization overhead. For

example, here is a class using a TreeMap that is expected to be large and concurrently accessed.

```
 class RWDictionary {
   private final Map<String, Data> m = new TreeMap<String, Data>();
   private final ReentrantReadWriteLock rwl = new ReentrantReadWriteLock();
   private final Lock r = rwl.readLock();
   private final Lock w = rwl.writeLock();

   public Data get(String key) {
     r.lock();
     try { return m.get(key); }
     finally { r.unlock(); }
   }
   public String[] allKeys() {
     r.lock();
     try { return m.keySet().toArray(); }
     finally { r.unlock(); }
   }
   public Data put(String key, Data value) {
     w.lock();
     try { return m.put(key, value); }
     finally { w.unlock(); }
   }
   public void clear() {
     w.lock();
     try { m.clear(); }
     finally { w.unlock(); }
   }
 }
```

Implementation Notes

This lock supports a maximum of 65535 recursive write locks and 65535 read locks. Attempts to exceed these limits result in `Error` throws from locking methods.

Since:

1.5

See Also:

`Serialized Form`

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description

static class	ReentrantReadWriteLock.ReadLock	The lock returned by method readLock().
static class	ReentrantReadWriteLock.WriteLock	The lock returned by method writeLock().

Constructor Summary

Constructors

Constructor	Description
ReentrantReadWriteLock()	Creates a new ReentrantReadWriteLock with default (nonfair) ordering properties.
ReentrantReadWriteLock(boolean fair)	Creates a new ReentrantReadWriteLock with the given fairness policy.

Method Summary

Concrete Methods

Modifier and Type	Method	Descrip
protected Thread	getOwner()	Returns the thre that current owns th write lo or null not owned.
protected Collection<Thread>	getQueuedReaderThreads()	Returns collecti contain threads that ma

		be wait to acqu the rea lock.
protected **Collection**<**Thread**>	**getQueuedThreads**()	Returns collecti contain threads that ma be wait to acqu either t read or write lo
protected **Collection**<**Thread**>	**getQueuedWriterThreads**()	Returns collecti contain threads that ma be wait to acqu the writ lock.
int	**getQueueLength**()	Returns an estimat of the number threads waiting acquire either t read or write lo
int	**getReadHoldCount**()	Queries the number reentra read ho on this lock by the current thread.
int	**getReadLockCount**()	

		Queries the number read loc held for this loc
protected **Collection**<**Thread**>	**getWaitingThreads**(**Condition** condition)	Returns collecti contain those threads that ma be wait on the given conditi associa with th write lo
int	**getWaitQueueLength**(**Condition** condition)	Returns an estimat of the number threads waiting the give conditi associa with th write lo
int	**getWriteHoldCount**()	Queries the number reentra write holds o this loc by the current thread.
boolean	**hasQueuedThread**(**Thread** thread)	Queries whethe the give thread

		waiting acquire either t read or write lo
boolean	**hasQueuedThreads**()	Queries whethe any threads are waiting acquire the rea or write lock.
boolean	**hasWaiters**(**Condition** condition)	Queries whethe any threads are waiting the give conditio associa with th write lo
boolean	**isFair**()	Returns `true` if this loc has fairness set true
boolean	**isWriteLocked**()	Queries the writ lock is held by any thread.
boolean	**isWriteLockedByCurrentThread**()	Queries the writ lock is held by the

		current thread.
ReentrantReadWriteLock.ReadLock	readLock()	Returns the lock used fo reading
String	toString()	Returns string identify this loc as well its lock state.
ReentrantReadWriteLock.WriteLock	writeLock()	Returns the lock used fo writing

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

ReentrantReadWriteLock

```
public ReentrantReadWriteLock()
```

Creates a new ReentrantReadWriteLock with default (nonfair) ordering properties.

ReentrantReadWriteLock

```
public ReentrantReadWriteLock(boolean fair)
```

Creates a new ReentrantReadWriteLock with the given fairness policy.

Parameters:

fair - true if this lock should use a fair ordering policy

Method Detail

writeLock

```
public ReentrantReadWriteLock.WriteLock writeLock()
```

Description copied from interface: ReadWriteLock
Returns the lock used for writing.

Specified by:

`writeLock` in interface `ReadWriteLock`

Returns:

the lock used for writing

readLock

```
public ReentrantReadWriteLock.ReadLock readLock()
```

Description copied from interface: ReadWriteLock
Returns the lock used for reading.

Specified by:

`readLock` in interface `ReadWriteLock`

Returns:

the lock used for reading

isFair

```
public final boolean isFair()
```

Returns `true` if this lock has fairness set true.

Returns:

true if this lock has fairness set true

getOwner

```
protected Thread getOwner()
```

Returns the thread that currently owns the write lock, or `null` if not owned. When this method is called by a thread that is not the owner, the return value reflects a best-effort approximation of current lock status. For example, the owner may be momentarily `null` even if there are threads trying to acquire the lock but have not yet done so. This method is designed to facilitate

construction of subclasses that provide more extensive lock monitoring facilities.

Returns:

`the owner, or null if not owned`

getReadLockCount

```
public int getReadLockCount()
```

Queries the number of read locks held for this lock. This method is designed for use in monitoring system state, not for synchronization control.

Returns:

`the number of read locks held`

isWriteLocked

```
public boolean isWriteLocked()
```

Queries if the write lock is held by any thread. This method is designed for use in monitoring system state, not for synchronization control.

Returns:

`true if any thread holds the write lock and false otherwise`

isWriteLockedByCurrentThread

```
public boolean isWriteLockedByCurrentThread()
```

Queries if the write lock is held by the current thread.

Returns:

`true if the current thread holds the write lock and false otherwise`

getWriteHoldCount

```
public int getWriteHoldCount()
```

Queries the number of reentrant write holds on this lock by the current thread. A writer thread has a hold on a lock for each lock action that is not matched by an unlock action.

Returns:

the number of holds on the write lock by the current thread, or zero if the write lock is not held by the current thread

getReadHoldCount

```
public int getReadHoldCount()
```

Queries the number of reentrant read holds on this lock by the current thread. A reader thread has a hold on a lock for each lock action that is not matched by an unlock action.

Returns:

the number of holds on the read lock by the current thread, or zero if the read lock is not held by the current thread

Since:

1.6

getQueuedWriterThreads

```
protected Collection<Thread> getQueuedWriterThreads()
```

Returns a collection containing threads that may be waiting to acquire the write lock. Because the actual set of threads may change dynamically while constructing this result, the returned collection is only a best-effort estimate. The elements of the returned collection are in no particular order. This method is designed to facilitate construction of subclasses that provide more extensive lock monitoring facilities.

Returns:

the collection of threads

getQueuedReaderThreads

```
protected Collection<Thread> getQueuedReaderThreads()
```

Returns a collection containing threads that may be waiting to acquire the read lock. Because the actual set of threads may change dynamically while constructing this result, the returned collection is only a best-effort estimate. The elements of the returned collection are in no particular order. This method is designed to facilitate construction of subclasses that provide more extensive lock monitoring facilities.

Returns:

the collection of threads

hasQueuedThreads

```
public final boolean hasQueuedThreads()
```

Queries whether any threads are waiting to acquire the read or write lock. Note that because cancellations may occur at any time, a `true` return does not guarantee that any other thread will ever acquire a lock. This method is designed primarily for use in monitoring of the system state.

Returns:

`true if there may be other threads waiting to acquire the lock`

hasQueuedThread

```
public final boolean hasQueuedThread(Thread thread)
```

Queries whether the given thread is waiting to acquire either the read or write lock. Note that because cancellations may occur at any time, a `true` return does not guarantee that this thread will ever acquire a lock. This method is designed primarily for use in monitoring of the system state.

Parameters:

`thread - the thread`

Returns:

`true if the given thread is queued waiting for this lock`

Throws:

`NullPointerException - if the thread is null`

getQueueLength

```
public final int getQueueLength()
```

Returns an estimate of the number of threads waiting to acquire either the read or write lock. The value is only an estimate because the number of threads may change dynamically while this method traverses internal data structures. This method is designed for use in monitoring of the system state, not for synchronization control.

Returns:

`the estimated number of threads waiting for this lock`

getQueuedThreads

```
protected Collection<Thread> getQueuedThreads()
```

Returns a collection containing threads that may be waiting to acquire either the read or write lock. Because the actual set of threads may change dynamically while constructing this result, the returned collection is only a best-effort estimate. The elements of the returned collection are in no particular order. This method is designed to facilitate construction of subclasses that provide more extensive monitoring facilities.

Returns:

the collection of threads

hasWaiters

```
public boolean hasWaiters(Condition condition)
```

Queries whether any threads are waiting on the given condition associated with the write lock. Note that because timeouts and interrupts may occur at any time, a `true` return does not guarantee that a future `signal` will awaken any threads. This method is designed primarily for use in monitoring of the system state.

Parameters:

condition - the condition

Returns:

true if there are any waiting threads

Throws:

IllegalMonitorStateException - if this lock is not held

IllegalArgumentException - if the given condition is not associated with this lock

NullPointerException - if the condition is null

getWaitQueueLength

```
public int getWaitQueueLength(Condition condition)
```

Returns an estimate of the number of threads waiting on the given condition associated with the write lock. Note that because timeouts and interrupts may occur at any time, the estimate serves only as an upper bound on the actual number of waiters. This method is designed for use in monitoring of the system state, not for synchronization control.

Parameters:

condition - the condition

Returns:

the estimated number of waiting threads

Throws:

IllegalMonitorStateException - if this lock is not held

IllegalArgumentException - if the given condition is not associated with this lock

NullPointerException - if the condition is null

getWaitingThreads

protected Collection<Thread> getWaitingThreads(Condition condition)

Returns a collection containing those threads that may be waiting on the given condition associated with the write lock. Because the actual set of threads may change dynamically while constructing this result, the returned collection is only a best-effort estimate. The elements of the returned collection are in no particular order. This method is designed to facilitate construction of subclasses that provide more extensive condition monitoring facilities.

Parameters:

condition - the condition

Returns:

the collection of threads

Throws:

IllegalMonitorStateException - if this lock is not held

IllegalArgumentException - if the given condition is not associated with this lock

NullPointerException - if the condition is null

toString

public String toString()

Returns a string identifying this lock, as well as its lock state. The state, in brackets, includes the String "Write locks =" followed by the number of reentrantly held write locks, and the String "Read locks =" followed by the number of held read locks.

Overrides:

toString in class Object

Returns:

a string identifying this lock, as well as its lock state

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.